

SIMATS ENGINEERING**SAVEETHA INSTITUTE OF MEDICAL AND TECHNICAL SCIENCES****CHENNAI – 602105****Department of Computer Science and Engineering****ASSESSMENT TOOL 2 – SCHEMA ANALYSIS AND VIVA VOCE**

Course Code:	DSA05
Course Title:	Query Processing for Data Science With Real Time Applications
CO Assessed:	CO2
Assessment:	Tool 2 – Schema Analysis and Viva Voce
Weightage / Total Marks:	35 Marks
CO2 Statement:	Use Python basics and SQL libraries to connect to databases SQLite, MySQL, PostgreSQL and perform CRUD operations like Create, Read, Update, Delete. (BL6)
Student Name:	Pola Nandivardhan
Register Number:	192324270
Section / Batch:	2023
Date of Submission:	
Faculty In-charge:	Dr. B.SHYAMALA GOWRI
Project Mode:	Individual Submission

Assessment Rubric

Identification of Entities and Attributes (5 Marks)	Understanding of Relationships (5 Marks)	Analysis of Keys and Constraints (5 Marks)	Detection of Design Issues (10 Marks)	Viva Voce (5 Marks)

Code of Conduct

I, Pola Nandivardhan (Reg. No. 192324270), certify that this submission is my original work and that I have adhered to all guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature: _____

1. University Course Registration System

A student can enroll in multiple courses, and each course can have multiple students.

Given Schema

Given Schema (Tables)
Student(StudentID, Name, DepartmentID)
Department(DepartmentID, DepartmentName)
Course(CourseID, CourseName, FacultyID)
Faculty(FacultyID, FacultyName)
Enrollment(StudentID, CourseID, Semester)

A. Identification of Entities and Attributes

Entity	Primary Key	Other Attributes
Student	StudentID	Name, DepartmentID (FK)
Department	DepartmentID	DepartmentName
Course	CourseID	CourseName, FacultyID (FK)
Faculty	FacultyID	FacultyName
Enrollment	StudentID + CourseID + Semester (composite)	Semester

B. Understanding of Relationships

Department 1:N Student — one department has many students, but each student belongs to exactly one department.
 Faculty 1:N Course — one faculty member can teach many courses (assuming a course has a single owning faculty).
 Student M:N Course — a student can enroll in many courses and a course can have many students; this many-to-many relationship is resolved through the Enrollment associative (bridge) table, which converts the M:N relationship into two 1:N relationships: Student 1:N Enrollment and Course 1:N Enrollment.

C. Analysis of Keys and Constraints

Table	Primary Key	Foreign Key(s)	Constraint Notes
Student	StudentID	DepartmentID → Department	NOT NULL on DepartmentID; DepartmentID must exist in Department
Department	DepartmentID	—	DepartmentName should be UNIQUE
Course	CourseID	FacultyID → Faculty	FacultyID references a valid Faculty row
Faculty	FacultyID	—	—
Enrollment	(StudentID, CourseID,	StudentID → Student, CourseID → Course	Composite key prevents the same student enrolling twice

Table	Primary Key	Foreign Key(s)	Constraint Notes
	Semester)		in the same course in the same semester

D. Detection of Design Issues

- No Grade attribute in Enrollment, so the schema cannot record a student's performance in a course.
- No explicit UNIQUE/composite-key enforcement shown in the raw schema, which could allow duplicate enrollment rows for the same student, course and semester if not enforced at the database level.
- Course assumes a single FacultyID, so team-taught or multi-section courses cannot be modelled.
- No timestamp (EnrollmentDate) to record when the enrollment happened, useful for auditing add/drop history.
- No status field (e.g., Active/Dropped/Completed) to track the lifecycle of an enrollment.

E. Viva Voce

Q1. Why is the Enrollment table necessary instead of storing CourseID directly in the Student table?

Ans: Because the Student–Course relationship is many-to-many: one student takes many courses and one course has many students. A single CourseID column in Student can hold only one value per row, so it cannot represent multiple enrollments and would force data duplication (repeating the entire student row for every course) or loss of information. The Enrollment table is a bridge/associative table that stores one row per (Student, Course) pair, correctly modelling the M:N relationship in a normalized way and avoiding redundancy and update anomalies.

Q2. If the university wants to track grades for each enrolled course, how would you modify the schema?

Ans: Add a Grade attribute to the Enrollment table, since a grade is a property of the specific enrollment (a particular student in a particular course in a particular semester) rather than of the Student or Course entity alone: Enrollment(StudentID, CourseID, Semester, Grade). This keeps the grade tied to the correct composite key and avoids anomalies that would occur if grade were stored elsewhere.

2. Hospital Management System

Multiple patients can consult the same doctor.

Given Schema

Given Schema (Tables)
Patient(PatientID, Name, Age)
Doctor(DoctorID, DoctorName, Specialization)
Appointment(AppointmentID, PatientID, DoctorID, Date)
Prescription(PrescriptionID, AppointmentID, Medicine)

A. Identification of Entities and Attributes

Entity	Primary Key	Other Attributes
Patient	PatientID	Name, Age
Doctor	DoctorID	DoctorName, Specialization
Appointment	AppointmentID	PatientID (FK), DoctorID (FK), Date
Prescription	PrescriptionID	AppointmentID (FK), Medicine

B. Understanding of Relationships

Patient M:N Doctor — a patient can consult many doctors and a doctor can consult many patients; this relationship is resolved through the Appointment associative table, giving Patient 1:N Appointment and Doctor 1:N Appointment. Appointment 1:N Prescription (as currently modelled) — one appointment can be linked to prescription rows, though as written each Prescription row holds only a single Medicine value, which restricts a prescription to one medicine per row.

C. Analysis of Keys and Constraints

Table	Primary Key	Foreign Key(s)	Constraint Notes
Patient	PatientID	—	—
Doctor	DoctorID	—	—
Appointment	AppointmentID	PatientID → Patient, DoctorID → Doctor	PatientID and DoctorID must reference existing rows; Date should be NOT NULL
Prescription	PrescriptionID	AppointmentID → Appointment	AppointmentID must reference a valid Appointment

D. Detection of Design Issues

- Prescription can hold only one Medicine per row, so multiple medicines for one appointment require repeating AppointmentID across several Prescription rows — an early sign the Medicine detail should be split into its own table with a proper multiplicity.
- No Dosage, Quantity, or Duration attributes for the prescribed medicine.

- No constraint preventing double-booking — two appointments for the same doctor at the same date/time are not blocked.
- Age is stored instead of Date of Birth, so the value becomes stale and must be manually updated over time.
- No appointment status (Scheduled/Completed/Cancelled) is tracked.

E. Viva Voce

Q1. What problems might occur if prescription details are stored directly in the Appointment table?

Ans: Since one appointment can result in multiple prescribed medicines, storing Medicine directly in Appointment would force either repeating the entire appointment row (PatientID, DoctorID, Date) for every medicine — causing data redundancy and update anomalies — or limiting the appointment to a single medicine, causing loss of information. Keeping Prescription as a separate table avoids this repeating-group problem and keeps the Appointment entity clean.

Q2. How would the schema change if a prescription contains multiple medicines?

Ans: Introduce a separate PrescriptionDetails (or Medicine) table: PrescriptionDetails(PrescriptionDetailID, PrescriptionID, MedicineName, Dosage, Duration), with PrescriptionID as a foreign key to Prescription. This converts the Prescription–Medicine relationship into a proper 1:N relationship, so one PrescriptionID can have many medicine rows without duplicating prescription-level data.

3. Online Shopping System

Each order may contain multiple products.

Given Schema

Given Schema (Tables)
Customer(CustomerID, Name)
Product(ProductID, ProductName, Price)
Order(OrderID, CustomerID, OrderDate)
OrderDetails(OrderID, ProductID, Quantity)

A. Identification of Entities and Attributes

Entity	Primary Key	Other Attributes
Customer	CustomerID	Name
Product	ProductID	ProductName, Price
Order	OrderID	CustomerID (FK), OrderDate
OrderDetails	OrderID + ProductID (composite)	Quantity

B. Understanding of Relationships

Customer 1:N Order — one customer can place many orders, each order belongs to one customer. Order M:N Product — an order can contain many products and a product can appear in many orders; this is resolved via the OrderDetails associative table, giving Order 1:N OrderDetails and Product 1:N OrderDetails.

C. Analysis of Keys and Constraints

Table	Primary Key	Foreign Key(s)	Constraint Notes
Customer	CustomerID	—	—
Product	ProductID	—	Price should be NOT NULL, >= 0
Order	OrderID	CustomerID → Customer	OrderDate NOT NULL
OrderDetails	(OrderID, ProductID)	OrderID → Order, ProductID → Product	Composite key prevents the same product being listed twice in one order; Quantity should be > 0

D. Detection of Design Issues

- OrderDetails does not store the unit price at the time of purchase; because Price lives only in Product, a later price change would silently alter the historical value of past orders.
- No OrderStatus (Pending/Shipped/Delivered/Cancelled) attribute.
- No stored TotalAmount for the order — it must always be recalculated from OrderDetails, which is acceptable but should be a documented design decision.

- No inventory/stock attribute on Product to check availability before an order is placed.

E. Viva Voce

Q1. Why is the OrderDetails table required in this schema?

Ans: Because the Order-Product relationship is many-to-many — one order can include several products and one product can appear in several orders. OrderDetails is the junction table that stores one row per (Order, Product) combination along with the Quantity, correctly modelling the M:N relationship and keeping the design normalized.

Q2. What would happen if ProductID were stored directly in the Order table?

Ans: Each Order row could then reference only a single product, so an order with multiple products would either be impossible to represent or would require repeating the entire order row (same OrderID, CustomerID, OrderDate) once per product — causing redundancy, update anomalies, and loss of a clean per-product Quantity value. This violates normalization and is exactly the problem the OrderDetails table solves.

4. Library Management System

Members can borrow multiple books.

Given Schema

Given Schema (Tables)
Member(MemberID, Name)
Book(BookID, Title, Author)
Issue(MemberID, BookID, IssueDate, ReturnDate)

A. Identification of Entities and Attributes

Entity	Primary Key	Other Attributes
Member	MemberID	Name
Book	BookID	Title, Author
Issue	MemberID + BookID (as given) — see design issue below	IssueDate, ReturnDate

B. Understanding of Relationships

Member M:N Book — a member can borrow many books over time, and a book can be borrowed by many members over time. This relationship is resolved through the Issue associative table, giving Member 1:N Issue and Book 1:N Issue.

C. Analysis of Keys and Constraints

Table	Primary Key	Foreign Key(s)	Constraint Notes
Member	MemberID	—	—
Book	BookID	—	—
Issue	MemberID + BookID (composite, as given)	MemberID → Member, BookID → Book	IssueDate NOT NULL; ReturnDate nullable until the book is returned

D. Detection of Design Issues

- Using (MemberID, BookID) as the identifying key cannot distinguish two different borrow events if the same member borrows the same book again after returning it — a surrogate IssueID primary key is needed to uniquely identify each borrowing instance.
- No support for multiple physical copies of the same book (BookID alone cannot indicate which copy was issued).
- No DueDate or Fine/FineAmount attribute, so overdue charges cannot be calculated or stored.
- No Status attribute (Issued/Returned/Overdue) to quickly query currently-borrowed books.

E. Viva Voce

Q1. Explain the relationship between Member and Book through the Issue table.

Ans: Member and Book share a many-to-many relationship — a member can borrow several books, and a book can be borrowed by several members at different times. The Issue table acts as a bridge (junction) table: each row links one MemberID to one BookID along with IssueDate and ReturnDate, turning the M:N relationship into two manageable 1:N relationships (Member–Issue and Book–Issue).

Q2. How would you modify the schema to track overdue fines?

Ans: Add a surrogate primary key IssueID to Issue (so repeat borrowings of the same book by the same member can be distinguished), plus a DueDate attribute and a Fine/FineAmount attribute — for example Issue(IssueID, MemberID, BookID, IssueDate, DueDate, ReturnDate, FineAmount) — where FineAmount can be calculated from the gap between DueDate and the actual ReturnDate.

5. Banking System

One customer can have multiple accounts.

Given Schema

Given Schema (Tables)
Customer(CustomerID, Name)
Account(AccountNo, CustomerID, Balance)
Transaction(TransactionID, AccountNo, Amount, Type)

A. Identification of Entities and Attributes

Entity	Primary Key	Other Attributes
Customer	CustomerID	Name
Account	AccountNo	CustomerID (FK), Balance
Transaction	TransactionID	AccountNo (FK), Amount, Type

B. Understanding of Relationships

Customer 1:N Account — one customer can hold multiple accounts, but (as currently modelled) each account belongs to exactly one customer. Account 1:N Transaction — one account can have many transactions over time, and each transaction belongs to exactly one account.

C. Analysis of Keys and Constraints

Table	Primary Key	Foreign Key(s)	Constraint Notes
Customer	CustomerID	—	—
Account	AccountNo	CustomerID → Customer	Balance should default to 0 and stay ≥ 0 (or per account-type rules)
Transaction	TransactionID	AccountNo → Account	Amount NOT NULL; Type restricted to a defined set (e.g., Deposit/Withdrawal)

D. Detection of Design Issues

- No timestamp (Date/Time) attribute on Transaction, making it impossible to order transactions chronologically or produce a statement.
- Balance is stored redundantly on Account instead of being derivable purely from Transaction history, which risks the two going out of sync if not updated consistently within a single controlled operation.
- Account currently allows only a single CustomerID, so joint accounts (more than one owner) cannot be represented.
- No branch/account-type attribute to distinguish savings, current, or loan accounts.

E. Viva Voce

Q1. Why should transactions be stored separately from account details?

Ans: Because an account can have many transactions over its lifetime, while each transaction belongs to one account — a 1:N relationship. Storing transactions in their own table preserves a complete, append-only audit trail of every deposit/withdrawal without repeatedly rewriting or bloating the Account row, and it avoids the anomalies that would occur from trying to fit a variable number of transactions into fixed columns on Account.

Q2. How would you redesign the schema to support joint accounts?

Ans: Introduce a junction table, e.g., AccountHolder(AccountNo, CustomerID) with a composite primary key, and remove the single CustomerID column from Account. This converts the Customer–Account relationship from 1:N to M:N, allowing one account to be linked to multiple customers (joint holders) and one customer to still hold multiple accounts.

6. Employee Payroll System

Employees belong to departments and receive monthly salaries.

Given Schema

Given Schema (Tables)
Employee(EmployeeID, Name, DepartmentID)
Department(DepartmentID, DepartmentName)
Payroll(PayrollID, EmployeeID, Salary, Month)

A. Identification of Entities and Attributes

Entity	Primary Key	Other Attributes
Employee	EmployeeID	Name, DepartmentID (FK)
Department	DepartmentID	DepartmentName
Payroll	PayrollID	EmployeeID (FK), Salary, Month

B. Understanding of Relationships

Department 1:N Employee — one department has many employees, each employee belongs to one department.
Employee 1:N Payroll — one employee has many monthly payroll records, each payroll record belongs to one employee.

C. Analysis of Keys and Constraints

Table	Primary Key	Foreign Key(s)	Constraint Notes
Employee	EmployeeID	DepartmentID → Department	DepartmentID must reference a valid Department
Department	DepartmentID	—	DepartmentName should be UNIQUE
Payroll	PayrollID	EmployeeID → Employee	(EmployeeID, Month) should ideally be UNIQUE to prevent duplicate payroll runs

D. Detection of Design Issues

- No explicit UNIQUE constraint on (EmployeeID, Month), so duplicate payroll rows for the same employee and month are not prevented by the schema as given.
- Month is stored without a Year, making records ambiguous across multiple years.
- Salary in Payroll does not separate base pay, allowances, and deductions — reducing transparency and auditability of how the final figure was computed.
- No PaymentDate or PaymentStatus attribute.

E. Viva Voce

Q1. Why is Department information separated into a different table?

Ans: Because many employees share the same department, and separating Department avoids repeating DepartmentName for every employee row. This is basic normalization: department-level facts are stored once in the Department table and referenced by DepartmentID, so an update (e.g., renaming a department) needs to happen in exactly one place.

Q2. What redundancy issues would arise if department details were stored in Employee records?

Ans: DepartmentName would be duplicated across every employee belonging to that department, wasting storage. It would also cause update anomalies (renaming the department requires updating every employee row), insertion anomalies (a new department cannot be recorded until at least one employee is assigned to it), and deletion anomalies (removing the last employee in a department would silently lose all information about that department).

7. Hotel Reservation System

Rooms can be booked by different customers at different times.

Given Schema

Given Schema (Tables)
Customer(CustomerID, Name)
Room(RoomID, RoomType)
Reservation(ReservationID, CustomerID, RoomID, CheckIn, CheckOut)

A. Identification of Entities and Attributes

Entity	Primary Key	Other Attributes
Customer	CustomerID	Name
Room	RoomID	RoomType
Reservation	ReservationID	CustomerID (FK), RoomID (FK), CheckIn, CheckOut

B. Understanding of Relationships

Customer 1:N Reservation — one customer can make many reservations, each reservation belongs to one customer.
 Room 1:N Reservation — one room can be booked in many separate reservations over time (at non-overlapping periods), each reservation is for one room. Customer and Room therefore share an M:N relationship over time, resolved through the Reservation associative table.

C. Analysis of Keys and Constraints

Table	Primary Key	Foreign Key(s)	Constraint Notes
Customer	CustomerID	—	—
Room	RoomID	—	—
Reservation	ReservationID	CustomerID → Customer, RoomID → Room	CheckIn < CheckOut; no schema-level check yet prevents overlapping bookings for the same RoomID

D. Detection of Design Issues

- No constraint (unique index, check constraint, or trigger) preventing two reservations for the same RoomID with overlapping CheckIn–CheckOut date ranges, allowing double booking.
- No ReservationStatus (Confirmed/Cancelled/CheckedOut) attribute.
- No Rate/Price attribute tied to the reservation or room type.
- No attribute capturing number of guests or special requests.

E. Viva Voce

Q1. Why should reservation details be maintained separately from room details?

Ans: A room's static attributes (RoomID, RoomType) do not change per booking, while a room can be reserved many times by different customers on different dates — a 1:N relationship. Keeping Reservation separate avoids repeating RoomType for every booking and lets each room accumulate an independent history of reservations without bloating the Room table.

Q2. How would you prevent double booking of rooms using the schema?

Ans: Add a database-level constraint or trigger that rejects a new Reservation if its (RoomID, CheckIn, CheckOut) range overlaps an existing Reservation for the same RoomID — for example, an exclusion constraint on date ranges (as supported by PostgreSQL) or an application/service-layer check before insert, combined with a ReservationStatus field so cancelled reservations do not block new bookings.

8. Food Delivery Application

Orders are delivered by delivery agents.

Given Schema

Given Schema (Tables)
Customer(CustomerID, Name)
Restaurant(RestaurantID, Name)
Order(OrderID, CustomerID, RestaurantID)
DeliveryAgent(AgentID, Name)
Delivery(OrderID, AgentID)

A. Identification of Entities and Attributes

Entity	Primary Key	Other Attributes
Customer	CustomerID	Name
Restaurant	RestaurantID	Name
Order	OrderID	CustomerID (FK), RestaurantID (FK)
DeliveryAgent	AgentID	Name
Delivery	OrderID + AgentID (as given)	—

B. Understanding of Relationships

Customer 1:N Order — one customer can place many orders. Restaurant 1:N Order — one restaurant can receive many orders. Order and DeliveryAgent share, as currently modelled, effectively a 1:1 relationship through Delivery (one order assigned to one agent); DeliveryAgent 1:N Delivery — one agent can handle many deliveries over time.

C. Analysis of Keys and Constraints

Table	Primary Key	Foreign Key(s)	Constraint Notes
Customer	CustomerID	—	—
Restaurant	RestaurantID	—	—
Order	OrderID	CustomerID → Customer, RestaurantID → Restaurant	—
DeliveryAgent	AgentID	—	—
Delivery	(OrderID, AgentID) as given	OrderID → Order, AgentID → DeliveryAgent	As written, OrderID is not guaranteed unique in Delivery, so multiple agents per order are not properly constrained either way

D. Detection of Design Issues

- Delivery has no dedicated surrogate primary key (e.g., DeliveryID), which limits clean tracking of delivery status history or reassignment.
- No DeliveryStatus (Assigned/PickedUp/Delivered) or timestamp attributes.
- Order table has no OrderDate, OrderStatus, or item-level details.
- The current design implicitly assumes one agent per order; it does not explicitly enforce this with a unique constraint.

E. Viva Voce

Q1. Why is Delivery maintained as a separate table?

Ans: Delivery captures the assignment relationship between an Order and a DeliveryAgent, which is conceptually distinct from the order-placement details (customer, restaurant). Keeping it separate lets a delivery be assigned, reassigned, or updated (e.g., status changes) independently of the Order and Restaurant data, and lets one DeliveryAgent be linked to many Delivery rows without duplicating Order or Customer information.

Q2. How would the schema change if multiple delivery agents could handle a single order?

Ans: Add a surrogate primary key DeliveryID to the Delivery table and allow multiple rows with the same OrderID, e.g., Delivery(DeliveryID, OrderID, AgentID, Stage) where Stage might indicate the leg of delivery (e.g., pickup vs. drop-off). This changes the Order–DeliveryAgent relationship from 1:1 to M:N, correctly modelled through the Delivery junction table.

9. Learning Management System (LMS)

Students can enroll in many courses.

Given Schema

Given Schema (Tables)
Student(StudentID, Name)
Course(CourseID, CourseName)
Enrollment(StudentID, CourseID)
Assignment(AssignmentID, CourseID)

A. Identification of Entities and Attributes

Entity	Primary Key	Other Attributes
Student	StudentID	Name
Course	CourseID	CourseName
Enrollment	StudentID + CourseID (composite)	—
Assignment	AssignmentID	CourseID (FK)

B. Understanding of Relationships

Student M:N Course — resolved through the Enrollment bridge table, giving Student 1:N Enrollment and Course 1:N Enrollment. Course 1:N Assignment — one course can have many assignments, each assignment belongs to one course.

C. Analysis of Keys and Constraints

Table	Primary Key	Foreign Key(s)	Constraint Notes
Student	StudentID	—	—
Course	CourseID	—	—
Enrollment	(StudentID, CourseID)	StudentID → Student, CourseID → Course	Composite key prevents duplicate enrollment in the same course
Assignment	AssignmentID	CourseID → Course	—

D. Detection of Design Issues

- No entity/table to record which student submitted which assignment or what grade they received — currently there is no link between Student and Assignment.
- Enrollment has no EnrollmentDate to know when a student joined a course.
- Assignment has no DueDate, MaxMarks, or description attributes.
- No Grade attribute anywhere in the schema, so overall course performance cannot be tracked.

E. Viva Voce

Q1. Why is Enrollment considered a bridge table?

Ans: Because Student and Course share a many-to-many relationship (a student can take many courses, and a course can have many students), and a relational table cannot directly store an M:N relationship. Enrollment sits between the two entities, holding only their primary keys (StudentID, CourseID) as foreign keys, effectively 'bridging' the two 1:N relationships (Student–Enrollment and Course–Enrollment) that together represent the original M:N relationship.

Q2. How would you modify the schema to record assignment grades?

Ans: Add a Submission table that links a Student to an Assignment along with their result: Submission(SubmissionID, AssignmentID, StudentID, Grade, SubmissionDate), with AssignmentID and StudentID as foreign keys. This models the fact that each student's grade is specific to a particular assignment, avoiding an attempt to store grades directly on Student, Course, or Assignment.

10. Bus Ticket Reservation System

Passengers can make multiple bookings.

Given Schema

Given Schema (Tables)
Passenger(PassengerID, Name)
Bus(BusID, Route)
Booking(BookingID, PassengerID, BusID, TravelDate)

A. Identification of Entities and Attributes

Entity	Primary Key	Other Attributes
Passenger	PassengerID	Name
Bus	BusID	Route
Booking	BookingID	PassengerID (FK), BusID (FK), TravelDate

B. Understanding of Relationships

Passenger 1:N Booking — one passenger can make many bookings, each booking (as currently modelled) belongs to one passenger. Bus 1:N Booking — one bus can appear in many bookings across different travel dates, each booking is for one bus.

C. Analysis of Keys and Constraints

Table	Primary Key	Foreign Key(s)	Constraint Notes
Passenger	PassengerID	—	—
Bus	BusID	—	—
Booking	BookingID	PassengerID → Passenger, BusID → Bus	No current constraint limits seats sold per (BusID, TravelDate) to bus capacity

D. Detection of Design Issues

- No SeatNumber attribute, so two bookings could unintentionally be assigned the same seat on the same bus and date.
- Booking currently links to only one PassengerID, so a single booking transaction covering a group of passengers cannot be represented without duplicating BusID/TravelDate rows.
- No Fare/Price attribute.
- No BookingStatus (Confirmed/Cancelled) attribute.

E. Viva Voce

Q1. Why should booking details be stored separately from passenger information?

Ans: A passenger's personal details (Name) do not change with every trip, while a passenger can make many bookings over time — a 1:N relationship. Keeping Booking separate avoids repeating passenger information for every trip and

lets each booking independently capture trip-specific data (BusID, TravelDate) without bloating or duplicating the Passenger table.

Q2. If a booking contains multiple passengers, how would you redesign the schema?

Ans: Introduce a junction table, e.g., BookingPassenger(BookingID, PassengerID, SeatNo) with a composite primary key, and remove the single PassengerID column from Booking (which would then just represent trip-level data such as BusID and TravelDate). This converts the Booking–Passenger relationship from 1:1 to M:N, allowing one booking to cover multiple passengers, each with their own seat.